MATLAB Vibrations Lab Plotting Scripts

Chat transcript, 21–27 October 2026 (long code listings replaced with "code output")

You
(Attached: Vibrations_datasets.zip.) Set up MATLAB code to load data from the "MAE Vibrations" folder, go into the "Vibrations datasets" zip, then the M003 folder. Files are named like "10_8_2026m003_hammer_x_ydat.txt", "..._x_yfft.txt" and "Shakerx_inch_y.txt" (x = 16, 20 or 24 in; y = a/b). Let the user type only the length and experiment. Make an amplitude vs. frequency plot with uncertainties and error bars, plus a theoretical curve: R times omega^2, scaled by the ratio of the measured maximum to that curve's maximum.

Claude
Wrote **plotVibration.m**, called as plotVibration('dat'|'fft'|'shaker', 16|20|24, 'a'|'b').

- Finds the folder, extracts the zip the first time, and locates the file by pattern.
- Hammer "dat" files are frequency data and "fft" files are time series (an FFT is taken). Column 2 is the beam output, column 3 the hammer input.
- Theory R = 1/sqrt((1-r^2)^2+(2 zeta r)^2), times omega^2, scaled to the measured peak.
- Asked for: uncertainty values, the theory formula, shaker units. Flagged the 20 in B run as odd.

You
(Attached: Vibrations_Lab_Procedure.pdf.) This should have all the necessary formulas and values. For any missing values use standard published values and cite the website.

Claude
Rebuilt the theory around the procedure:

- fn from beam theory: k = 3EI/L^3, equivalent mass 33/140 of the beam mass (eqs 3, 4), with propagated uncertainty.
- zeta from log decrement of the hammer time series (eqs 5, 6). Measured fn via eq 7 (hammer) and eq 8 (shaker).
- Cited values: MatWeb AISI 1018 (E = 200 GPa, 7.87 g/cc), Total Materia (ranges), NI-9234 specs (0.34 % gain error), McMaster bar tolerances, US Patent 6,526,835 (0.2357 M), McGill ECSE 304 notes (magnitude ratio).
- Noted measured fn is about 8–13 % below theory (clamp compliance).

You
make the code copy and pasteable

Claude
Turned it into a plain script (plotVibrationData.m) with the file type, length and experiment set at the top. Pasted the full script.

[code output]

You
Now I need Amplitude vs Normalized Frequency plots for the output for each length of beam, with uncertainties. Same plot, but the x-axis divided by the theoretical undamped natural frequency.

Claude
Added normalizeFreq (x = f/fn,theory, with fn uncertainty in the horizontal error bars) and a loop over all three lengths.

[code output]

You
I want it to function like the last code, one plot at a time. The beam is 2.07 lb, 0.9985 in wide, 29.5 in long, 0.288 in thick, carbon steel.

Claude

Back to one plot per run, with the measured beam and the mass of the vibrating part = (L/29.5) x 2.07 lb.

- New theory fn: 39.50 ± 0.99, 25.28 ± 0.63, 17.56 ± 0.44 Hz.
- Flagged: a 29.5 x 0.9985 x 0.288 in steel bar should weigh about 2.41 lb; 2.07 lb fits 0.25 in thickness.

[code output]

You
Now I need a plot of undamped natural and resonance frequencies for my various lengths (with error bars), using the experimental damping ratio from the hammer test, with uncertainties.

Claude
New script plotNaturalFrequencies.m: theory, hammer and shaker fn and fr vs. length with propagated uncertainties, plus a panel for fn – fr (under 1 mHz because zeta is about 0.002–0.003).

You
make it copy and pasteable always

Claude
Pasted the full script, and will paste code in every reply from now on.

[code output]

You
does this plot all three lengths at the same time? i need it to

Claude
Yes. One run plots 16, 20 and 24 in on one figure.

You
i need the plot to have error bars and i need the plot to read fr and fn

Claude
Split into labelled f_n and f_r panels with bigger error bars and a value ± uncertainty label on every point.

[code output]

You
it needs to be one plot

Claude
Put everything on one axes: filled markers for fn, open markers for fr, side by side at each length, with error bars and labels.

[code output]

You
Now I need a code that makes 4 subplots in one figure: voltage vs time (input and output) and amplitude vs frequency (input and output). Only one length, typed as 16, 20 or 24.

Claude
New script plotHammerSignals.m: a 2 x 2 figure with the hammer strike and the output peak labelled.

[code output]

You
Now 2 subplots in 1 figure: magnitude of frequency response |H(f)| vs frequency and phase lag vs frequency. y-axis labels |H(f)| and Phase lag. Let the user pick the hammer length.

Claude
New script plotFrequencyResponse.m: H = FFT(output)/FFT(input) from the hammer time series, with an SDOF theory overlay.

- The phase is shown for displacement by default (0 → 90 → 180 deg); the raw accelerometer phase is optional.

  [code output]

You
Now source data from the "Lab Data V" folder: files "10_8_2026jamesXvY.txt", X = experiment 1–14, Y = dat or fft. I need 1 figure per parameter change (length, damping coefficient, end mass, material), each with |H(f)| and phase lag subplots.

Claude
Asked for the files and the run-to-parameter mapping (proposed runs 1–3 length, 4–8 damping, 9–11 end mass, 12–14 material).

You
(Attached: Lab_Data_V.zip.) Everything you said about the runs is correct.

Claude
New script plotSimulationResponse.m: "fft" holds |H(f)| and "dat" holds the phase angle. One figure per parameter, or customRuns.

- Flagged recording problems: peak heights match theory but several runs' frequency axes match the previous run (the simulation must be run twice per setting); run 3 looks like c = 9 kg/s; runs 4–8 resonate at 1.58 Hz.

  [code output]

You
Instead of run on the plot I need it to say the length used or the material type etc.

Claude
Legends now show the case (L = 16 in, c = 1 kg/s, m_end = 0.25 kg, Aluminum).

[code output]

You
(Attached: undampednatresonance.m, twosubplotqfive.m, ShakerTestCode.m, shakernormalizedfreq.m, hammerquadplot.m.) Put all of these together without losing any functionality, and make the code as short as possible.

Claude
Combined them into **vibrationLab.m** (479 lines, compared with 2,429 for the five scripts). plotType selects 'amplitude', 'natres', 'frf' or 'quad'. Shared code is written once and every setting is kept. The measured beam values are used; the old 1.00 x 0.25 in values can be restored in the settings.

[code output]

You
(Attached: part6plots.m.) Merge that code with this code, keeping the function the same and the code short.

Claude
Added the simulation plots as plotType = 'sim' (569 lines in total). The sim frequency limit is now simFreqLim, and 'sim' only needs the Lab Data V folder.

[code output]

You
make this chat a pdf. any areas with extremely long code just say "code output"

Claude
Created this PDF.

vibrationLab.m

Combined MATLAB script for the cantilever-beam vibrations lab: plotType = 'amplitude', 'natres', 'frf', 'quad' or 'sim'. 569 lines.

```
% VIBRATIONLAB  All cantilever-beam vibration lab plots in one script.
% Paste into a new MATLAB script (e.g. vibrationLab.m), set CHOOSE THE PLOT, press Run.
%   'amplitude' : amplitude vs. (normalized) frequency of one file, with error bars and
%                 a theoretical R*w^2 curve scaled to the measured maximum
%   'natres'    : undamped natural (fn) and resonance (fr) frequency vs. beam length
%   'frf'       : |H(f)| and phase lag vs. frequency of the hammer test
%   'quad'      : hammer input/output, voltage vs. time and amplitude vs. frequency
%   'sim'       : simulation runs (Lab Data V), |H(f)| and phase lag, one figure per parameter
%
% Data: <mainFolder>\Vibrations datasets\<groupFolder> (the zip is extracted the first time)
%   hammer 'dat': col 1 frequency [Hz], col 2 beam output, col 3 hammer input
%   hammer 'fft': col 1 time [s],       col 2 beam output, col 3 hammer input
%   shaker      : col 2 frequency [Hz], col 3 output amplitude (col 4 ignored)
% Simulation: <mainFolder>\Lab Data V\10_8_2026jamesXvY.txt, X = run 1-14, Y = 'dat'/'fft'
%   fft: col 2 = |H(f)|, dat: col 2 = phase angle [deg]; phase lag = -phase angle
%   runs 1-3 length, 4-8 damping, 9-11 end mass, 12-14 material (procedure section 3)
% Theory (Vibrations Lab Procedure): k = 3EI/L^3, meq = 33/140 m + m_end, fn = sqrt(k/meq)/2pi
%   (eqs 3, 4); zeta by log decrement of the hammer time series (eqs 5, 6);
%   fn = fd/sqrt(1-zeta^2) (eq 7, hammer), fr = fn*sqrt(1-2zeta^2) (eq 8, shaker);
%   R = 1/sqrt((1-r^2)^2 + (2 zeta r)^2), r = f/fn. Uncertainties: root-sum-square.
% Sources: [1] MatWeb AISI 1018 cold drawn, E = 200 GPa, 7.87 g/cc
%   https://matweb.com/search/datasheettext.aspx?matid=6817
%   [2] Total Materia AISI 1018, E = 190-210 GPa, 7.67-7.86 g/cc  https://www.totalmateria.com/en-us/material/1250012
%   [3] NI-9234 gain error 0.34 % max  https://ni.com/docs/en-US/bundle/ni-9234-specs
%   [4] Meff = 0.2357 M (33/140), US Patent 6,526,835
%   https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/6526835
%   [5] Second-order magnitude ratio, McGill ECSE 304 L30  https://cim.mcgill.ca/~boulet/304-303B/1999/L30.pdf

clear; clc; close all;

%% ====================== CHOOSE THE PLOT ======================
plotType      = 'amplitude';   % 'amplitude', 'natres', 'frf', 'quad' or 'sim'
beamLength    = 24;            % 16, 20 or 24 in (all plots except 'natres')
experiment    = 'b';           % 'a' or 'b'
fileType      = 'shaker';      % 'amplitude' only: 'dat', 'fft' or 'shaker'
normalizeFreq = true;          % 'amplitude' only: x-axis = f / fn,theory
beamLengths   = [16 20 24];    % 'natres' only
parameter     = 'material';    % 'sim' only: 'length', 'damping', 'endmass', 'material' or 'all'
customRuns    = [12 13 14];    % 'sim' only: any runs on one figure; [] = use 'parameter'

%% ====================== USER SETTINGS ======================
mainFolder    = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Vibrations';
zipName       = 'Vibrations datasets.zip';   % zip file inside mainFolder
datasetFolder = 'Vibrations datasets';       % folder inside the zip
groupFolder   = 'M003';                      % group folder inside datasetFolder

% Beam (measured, carbon steel). Set totalMass = [] to use rho instead.
lb2kg = 0.45359237;
beam = struct('width', 0.9985, 'thick', 0.288, 'totalMass', 2.07*lb2kg, 'totalLength', 29.5, ...
              'E', 200e9, 'rho', 7870, 'endMass', 0, ...          % [in, in, kg, in, Pa, kg/m^3, kg]  [1]
              'uLength', 1/32, 'uTotalLength', 1/32, ...          % half of 1/16 in tape division
              'uWidth', 0.0005, 'uThick', 0.0005, ...             % half of 0.001 in caliper division
```

```
            'uTotalMass', 0.005*lb2kg, 'uE', 10e9, 'uRho', 100); % scale; [2]; [1,2]

% Theory and damping
theoryFnSource = 'beam';       % 'amplitude': fn from 'beam' theory or the 'measured' peak
frfFnSource    = 'measured';   % 'frf':       fn from 'beam' theory or the 'measured' peak
theoryFn   = [];               % 'amplitude': override fn [Hz]
theoryZeta = [];               % 'amplitude': override zeta
showTheory = true;             % 'frf': draw the theoretical curve
phaseRef   = 'displacement';   % 'frf': 'displacement' (0 -> 90 -> 180 deg) or 'acceleration'
logDecCycles = 10;             % n in eq 6
logDecSkip   = 2;              % peaks skipped after the hammer strike

% Measurement uncertainties
daqGainPct    = 0.34;          % NI 9234 gain error [%]  [3]
accelPct      = 0;             % accelerometer sensitivity [%] (add if known)
shakerAmpRes  = 0.0005;        % half of the 0.001 last digit
shakerFreqUnc = 0.5;           % shaker frequency [Hz] (1 Hz steps); hammer: half the FFT bin
nErrorBars    = 40;            % hammer files: points that get error bars

% Plot options
freqLim        = [];           % 'amplitude'/'frf' [fmin fmax] Hz; [] = automatic
quadFreqLim    = [0 250];      % 'quad' [fmin fmax] Hz; [] = up to Nyquist
timeLim        = [];           % 'quad' [tmin tmax] s; [] = whole record
showTimeSeries = true;         % 'amplitude' with 'fft' files: also plot the time series
voltUnit       = 'V\times10^2';% 'quad' y-axis unit (from the file headers)

% Simulation ('sim')
simFolder    = 'Lab Data V';   % folder inside mainFolder
simZip       = 'Lab Data V.zip';% used only if the folder is missing
magType      = 'fft';          % file type (Y) with |H(f)|
phaseType    = 'dat';          % file type (Y) with the phase angle [deg]
logMagnitude = true;           % log scale for |H(f)|
simFreqLim   = [];             % [fmin fmax] Hz for every figure; [] = automatic
groups = struct('key', {'length', 'damping', 'endmass', 'material'}, ...
  'title', {'Varying Beam Length', 'Varying Damping Coefficient', 'Varying End Mass', 'Varying Material'}, ...
  'runs',  {[1 2 3], [4 5 6 7 8], [9 10 11], [12 13 14]}, ...
  'labels', {{'L = 16 in', 'L = 20 in', 'L = 24 in'}, ...
             {'c = 1 kg/s', 'c = 3 kg/s', 'c = 5 kg/s', 'c = 7 kg/s', 'c = 9 kg/s'}, ...
             {'m_{end} = 0.25 kg', 'm_{end} = 0.40 kg', 'm_{end} = 0.65 kg'}, ...
             {'Carbon steel', 'Stainless steel', 'Aluminum'}});
%% ============================================================

experiment = lower(char(experiment));  fileType = lower(char(fileType));  plotType = lower(char(plotType));
assert(ismember(experiment, {'a','b'}), 'Experiment must be ''a'' or ''b''.');
assert(ismember(plotType, {'natres','sim'}) || ismember(beamLength, [16 20 24]), 'Length must be 16, 20 or 24 (inches).');
if ~strcmp(plotType, 'sim'), dataDir = findDataFolder(mainFolder, zipName, datasetFolder, groupFolder); end
opt  = {'FileType', 'text', 'Delimiter', '\t'};
blue = [0 0.3 0.7];  red = [0.85 0.1 0.1];  green = [0.1 0.55 0.2];

switch plotType
case 'amplitude' % ===== amplitude vs. (normalized) frequency of one file =====
    assert(ismember(fileType, {'dat','fft','shaker'}), 'fileType must be ''dat'', ''fft'' or ''shaker''.');
    file = findDataFile(dataDir, fileType, beamLength, experiment);
    [~, fName, fExt] = fileparts(file);  fprintf('\nUsing file: %s\n', file);
    [fnBeam, uFnBeam] = beamNaturalFrequency(beamLength, beam);
    relPct = hypot(daqGainPct, accelPct);
    isHammer = ~strcmp(fileType, 'shaker');
```

```
if isHammer
    M = readmatrix(file, opt{:}, 'NumHeaderLines', 1);
    if isTimeFile(file)        % time series -> single-sided spectrum of the beam output
        tSig = M(:,1);  ySig = M(:,2);  N = numel(ySig);
        A = abs(fft(ySig - mean(ySig)))/N;  A = A(1:floor(N/2)+1);  A(2:end-1) = 2*A(2:end-1);
        f = (0:floor(N/2))'/(N*mean(diff(tSig)));
    else                       % frequency data: drop the zero padding above Nyquist
        last = find(M(:,2) ~= 0, 1, 'last');  f = M(1:last,1);  A = M(1:last,2);
    end
    uF = (f(2)-f(1))/2*ones(size(f));
    uA = hypot(median(A(2:end)), relPct/100*A);           % noise floor + gain error
    ampLabel = 'Amplitude (V\times10^2)';
    win = find(f >= 0.5*fnBeam & f <= 1.5*fnBeam);  [Apk, k] = max(A(win));  iPk = win(k);
else
    [f, A] = readShaker(file);
    uF = shakerFreqUnc*ones(size(f));  uA = hypot(shakerAmpRes, relPct/100*A);
    ampLabel = 'Output Amplitude';
    [Apk, iPk] = max(A);
end
fPk = f(iPk);
if isempty(theoryZeta)
    [zeta, uZeta, zetaSrc] = dampingFromHammer(dataDir, beamLength, experiment, fnBeam, logDecCycles, logDecSkip);
    if isnan(zeta)
        if ~isHammer
            error('Could not get zeta from the %d in hammer data by log decrement. Set theoryZeta in USER SETTINGS.', beamLength);
        end
        zeta = halfPowerZeta(f, A, iPk);  uZeta = 0;
        zetaSrc = 'half-power bandwidth of this file (log decrement failed)';
    end
else
    zeta = theoryZeta;  uZeta = 0;  zetaSrc = 'user setting';
end
fnMeas = fPk/sqrt(1 - (2 - isHammer)*zeta^2);   % eq 7 (hammer) or eq 8 (shaker)
uFnMeas = uF(iPk);
if ~isempty(theoryFn),                      fn = theoryFn; uFn = 0;       fnSrc = 'user setting';
elseif strcmpi(theoryFnSource, 'measured'), fn = fnMeas;   uFn = uFnMeas; fnSrc = 'measured peak';
else,                                       fn = fnBeam;   uFn = uFnBeam; fnSrc = 'beam theory';
end
if isempty(freqLim)
    if isHammer, freqLim = [0, 3*max(fn, fPk)];
    else
        lo = min([f; fn-uFn]);  hi = max([f; fn+uFn]);  pad = max(2, 0.15*(hi-lo));
        freqLim = [max(0, lo-pad), hi+pad];
    end
end
if normalizeFreq   % fn,theory uncertainty goes into the horizontal error bars instead of the band
    xS = fnBeam;  uX = hypot(uF/fnBeam, f*uFnBeam/fnBeam^2);  xLbl = 'Normalized Frequency  f / f_{n,theory}';
    uFnBand = uFn*~strcmp(fnSrc, 'beam theory');  pType = 'Normalized Frequency';
else
    xS = 1;  uX = uF;  xLbl = 'Frequency (Hz)';  uFnBand = uFn;  pType = 'Frequency';
end
fnSet = fn + uFnBand*[0 -1 1 -1 1];  zSet = max(zeta + uZeta*[0 -1 -1 1 1], 1e-4);
fT = theoryGrid(freqLim, fnSet, zSet, 2000);
theoAll = zeros(numel(fT), 5);
for i = 1:5, theoAll(:,i) = theoryCurve(fT, fnSet(i), zSet(i), Apk); end
[theo, scale] = theoryCurve(fT, fn, zeta, Apk);
pctDiff = 100*(fnMeas - fnBeam)/fnBeam;
```

```
    fprintf(['Theoretical fn (beam)   = %.2f +/- %.2f Hz\nMeasured peak           = %.4g at %.2f Hz\n' ...
             'Measured fn (eq %d)      = %.2f +/- %.2f Hz  (%+.1f %% vs. theory)\n' ...
             'Damping ratio zeta       = %.4f +/- %.4f  (%s)\nTheory curve: fn = %.2f Hz (%s), scale factor = %.4g\n'], ...
            fnBeam, uFnBeam, Apk, fPk, 7 + ~isHammer, fnMeas, uFnMeas, pctDiff, zeta, uZeta, zetaSrc, fn, fnSrc, scale);
    if normalizeFreq
        fprintf('Measured fn / fn,theory = %.3f +/- %.3f\n', fnMeas/fnBeam, hypot(uFnMeas/fnBeam, fnMeas*uFnBeam/fnBeam^2));
    end

    if isHammer, ttl = sprintf('Hammer %s: %d in, experiment %s', upper(fileType), beamLength, upper(experiment));
    else,        ttl = sprintf('Shaker: %d in, experiment %s', beamLength, upper(experiment)); end
    twoTiles = strcmp(fileType, 'fft') && showTimeSeries;
    newFig(ttl, [100 100 900 500 + 250*twoTiles]);
    if twoTiles
        tl = tiledlayout(2, 1, 'TileSpacing', 'compact');  title(tl, ttl, 'Interpreter', 'none');
        nexttile;  plot(tSig, ySig, 'LineWidth', 1);  axis tight;
        lab('Time (s)', 'Beam output (V\times10^2)', 'Time Series');  nexttile;
    end
    hold on;
    fill([fT; flipud(fT)]/xS, [max(theoAll,[],2); flipud(min(theoAll,[],2))], red, 'EdgeColor', 'none', ...
         'FaceAlpha', 0.15, 'DisplayName', 'Theory uncertainty (f_n, \zeta)');
    if isHammer
        ir = find(f >= freqLim(1) & f <= freqLim(2));
        fill([f(ir); flipud(f(ir))]/xS, [A(ir)+uA(ir); flipud(max(A(ir)-uA(ir), 0))], blue, ...
             'EdgeColor', 'none', 'FaceAlpha', 0.25, 'DisplayName', 'Measurement uncertainty');
        plot(f(ir)/xS, A(ir), '-', 'Color', blue, 'LineWidth', 1.2, 'DisplayName', 'Measured');
        eb = unique([ir(1:max(1, round(numel(ir)/nErrorBars)):end); iPk]);
        errorbar(f(eb)/xS, A(eb), uA(eb), uA(eb), uX(eb), uX(eb), 'o', 'Color', blue, 'MarkerSize', 4, ...
                 'MarkerFaceColor', blue, 'CapSize', 4, 'DisplayName', 'Error bars');
    else
        errorbar(f/xS, A, uA, uA, uX, uX, 'o', 'Color', blue, 'MarkerSize', 6, 'MarkerFaceColor', blue, ...
                 'LineWidth', 1.2, 'CapSize', 6, 'DisplayName', 'Measured');
    end
    plot(fT/xS, theo, '--', 'Color', red, 'LineWidth', 1.5, ...
         'DisplayName', sprintf('Theory (f_n = %.2f Hz, \\zeta = %.4f)', fn, zeta));
    hold off;
    if twoTiles, t2 = ['Amplitude vs. ' pType]; else, t2 = {ttl, ['Amplitude vs. ' pType]}; end
    lab(xLbl, ampLabel, t2);  subtitle([fName fExt], 'Interpreter', 'none');  legend('Location', 'northeast');
    xlim(freqLim/xS);  ylim([0, 1.15*max([Apk + uA(iPk); theoAll(:)])]);
    s = {sprintf('f_{n,theory} = %.2f \\pm %.2f Hz', fnBeam, uFnBeam), ...
         sprintf('f_{n,measured} = %.2f \\pm %.2f Hz (%+.1f%%)', fnMeas, uFnMeas, pctDiff), ...
         sprintf('\\zeta = %.4f \\pm %.4f', zeta, uZeta)};
    if normalizeFreq, s{end+1} = sprintf('x-axis divided by f_{n,theory} = %.2f Hz', fnBeam); end
    text(0.02, 0.97, s, 'Units', 'normalized', 'VerticalAlignment', 'top', 'BackgroundColor', 'w', ...
         'EdgeColor', [0.7 0.7 0.7], 'FontSize', 9);

case 'natres' % ===== undamped natural and resonance frequency vs. length =====
    nL = numel(beamLengths);
    [zeta, uZeta, fnT, uFnT, frT, uFrT, fnH, uFnH, frH, uFrH, fnS, uFnS, frS, uFrS] = deal(NaN(1, nL));
    for j = 1:nL
        L = beamLengths(j);  fprintf('\n===== %d in =====\n', L);
        [fnT(j), uFnT(j)] = beamNaturalFrequency(L, beam);
        [zeta(j), uZeta(j), src] = dampingFromHammer(dataDir, L, experiment, fnT(j), logDecCycles, logDecSkip);
        if isnan(zeta(j)), warning('No usable hammer time series for %d in; skipping this length.', L); continue; end
        fprintf('zeta = %.4f +/- %.4f  (%s)\n', zeta(j), uZeta(j), src);
        [frT(j), uFrT(j)] = propagate(@(x) x(1)*sqrt(1 - 2*x(2)^2), [fnT(j) zeta(j)], [uFnT(j) uZeta(j)]);
        try   % hammer: damped peak fd -> fn (eq 7) -> fr (eq 8)
            file = findDataFile(dataDir, 'dat', L, experiment);
```

```
        M = readmatrix(file, opt{:}, 'NumHeaderLines', 1);
        win = find(M(:,1) >= 0.5*fnT(j) & M(:,1) <= 1.5*fnT(j));  [~, k] = max(M(win,2));
        fd = M(win(k),1);  uFd = (M(2,1) - M(1,1))/2;
        [fnH(j), uFnH(j)] = propagate(@(x) x(1)/sqrt(1 - x(2)^2), [fd zeta(j)], [uFd uZeta(j)]);
        [frH(j), uFrH(j)] = propagate(@(x) x(1)*sqrt(1 - 2*x(2)^2)/sqrt(1 - x(2)^2), [fd zeta(j)], [uFd uZeta(j)]);
        fprintf('Hammer peak fd = %.3f +/- %.3f Hz  (%s)\n', fd, uFd, file);
    catch err, warning('Hammer data for %d in skipped: %s', L, err.message);
    end
    try   % shaker: resonance peak fr -> fn (eq 8)
        file = findDataFile(dataDir, 'shaker', L, experiment);
        [fs, As] = readShaker(file);  [~, k] = max(As);  frS(j) = fs(k);  uFrS(j) = shakerFreqUnc;
        [fnS(j), uFnS(j)] = propagate(@(x) x(1)/sqrt(1 - 2*x(2)^2), [frS(j) zeta(j)], [uFrS(j) uZeta(j)]);
        fprintf('Shaker peak fr = %.3f +/- %.3f Hz  (%s)\n', frS(j), uFrS(j), file);
    catch err, warning('Shaker data for %d in skipped: %s', L, err.message);
    end
end
fprintf('\n%-6s %-18s %-18s %-18s %-18s %-18s %-18s %-18s\n', 'L [in]', 'zeta', 'fn theory [Hz]', ...
        'fr theory [Hz]', 'fn hammer [Hz]', 'fr hammer [Hz]', 'fn shaker [Hz]', 'fr shaker [Hz]');
for j = 1:nL
    fprintf('%-6d %-18s %-18s %-18s %-18s %-18s %-18s %-18s\n', beamLengths(j), pm(zeta(j), uZeta(j), 4), ...
            pm(fnT(j), uFnT(j), 3), pm(frT(j), uFrT(j), 3), pm(fnH(j), uFnH(j), 3), pm(frH(j), uFrH(j), 3), ...
            pm(fnS(j), uFnS(j), 3), pm(frS(j), uFrS(j), 3));
end
fprintf('\nfn - fr (theory) = fn*[1 - sqrt(1 - 2*zeta^2)]:\n');
fprintf('  %d in: %.4f mHz\n', [beamLengths; 1000*(fnT - frT)]);

uL = beam.uLength*ones(1, nL);  dx = 0.6;
Lc = linspace(min(beamLengths) - 3, max(beamLengths) + 3, 200);
[fnC, uFnC] = arrayfun(@(x) beamNaturalFrequency(x, beam), Lc);   % continuous theory fn(L)
okZ = ~isnan(zeta);
if nnz(okZ) >= 2, zC = interp1(beamLengths(okZ), zeta(okZ), Lc, 'nearest', 'extrap');
else, zC = max([zeta(okZ) 0])*ones(size(Lc)); end
frC = fnC.*sqrt(1 - 2*zC.^2);
newFig('Natural and resonance frequencies', [50 100 1400 600]);
tl = tiledlayout(1, 2, 'TileSpacing', 'compact');
title(tl, sprintf('Undamped Natural and Resonance Frequencies vs. Beam Length (experiment %s)', upper(experiment)));
subtitle(tl, ['\zeta from the hammer test (log decrement): ' strjoin(arrayfun(@(j) sprintf('%d in: %.4f \\pm %.4f', ...
         beamLengths(j), zeta(j), uZeta(j)), 1:nL, 'UniformOutput', false), ',   ')]);
panels = {'f_n', 'Undamped Natural Frequency  f_n', fnC, fnT, uFnT, fnH, uFnH, fnS, uFnS
          'f_r', 'Resonance Frequency  f_r = f_n\surd(1 - 2\zeta^2)', frC, frT, uFrT, frH, uFrH, frS, uFrS};
for p = 1:2
    [sym, ttl, yC, yT, uT, yH, uH, yS, uS] = panels{p, :};
    nexttile;  hold on;
    fill([Lc fliplr(Lc)], [yC + uFnC, fliplr(yC - uFnC)], red, 'EdgeColor', 'none', 'FaceAlpha', 0.12, ...
         'DisplayName', ['Theory ' sym '(L) \pm uncertainty']);
    plot(Lc, yC, '-', 'Color', red, 'LineWidth', 1, 'DisplayName', ['Theory ' sym '(L)']);
    % theory left (label above), hammer centre (label left, below), shaker right (label right)
    series = {-dx, yT, uT, red, 's', 'theory', 0, 1, 'center', 'bottom'
               0,  yH, uH, blue, 'o', 'hammer', -0.15, -1, 'right', 'top'
               dx, yS, uS, green, '^', 'shaker', 0.15, 0, 'left', 'middle'};
    for s = 1:3
        [off, y, u, col, mk, name, tdx, tdy, ha, va] = series{s, :};
        errorbar(beamLengths + off, y, u, u, uL, uL, mk, 'Color', col, 'MarkerFaceColor', col, ...
                 'MarkerSize', 6, 'LineWidth', 1.5, 'CapSize', 12, 'DisplayName', [sym ' ' name]);
        for j = find(~isnan(y))
            text(beamLengths(j) + off + tdx, y(j) + tdy*u(j), sprintf('%.2f \\pm %.2f', y(j), u(j)), 'Color', col, ...
                 'FontSize', 8, 'HorizontalAlignment', ha, 'VerticalAlignment', va);
```

```
            end
        end
        hold off;  lab('Beam Length (in)', [sym ' (Hz)'], ttl);
        xticks(beamLengths);  xlim([min(Lc) max(Lc)]);  legend('Location', 'northeast');
    end

case 'frf' % ===== |H(f)| and phase lag of the hammer test =====
    file = findDataFile(dataDir, 'fft', beamLength, experiment);  fprintf('Using file: %s\n', file);
    M = readmatrix(file, opt{:}, 'NumHeaderLines', 1);  t = M(:,1);  yOut = M(:,2);  xIn = M(:,3);
    N = numel(t);  nH = floor(N/2) + 1;  f = (0:nH-1)'/(N*mean(diff(t)));
    H = fft(yOut - mean(yOut))./fft(xIn - median(xIn));  H = H(1:nH);   % H = Y/X
    magH = abs(H);
    disp_ = strcmpi(phaseRef, 'displacement');
    lagLo = -270 + 180*disp_;                       % plot range [-90,270) or [-270,90)
    wrapLag = @(v) mod(v - lagLo, 360) + lagLo;
    lag = wrapLag(-angle(H)*180/pi + 180*disp_);    % a = -w^2 x: x lags a by 180 deg
    fnBeam = beamNaturalFrequency(beamLength, beam);
    win = find(f >= 0.5*fnBeam & f <= 1.5*fnBeam);  [Hpk, k] = max(magH(win));  iPk = win(k);  fPk = f(iPk);
    if isempty(freqLim), freqLim = [0, 3*fPk]; end
    zeta = logDecrement(t, yOut, xIn, fnBeam, logDecCycles, logDecSkip);
    if isnan(zeta) || zeta <= 0
        warning('Log decrement failed for this run; the theoretical curve is not drawn.');  showTheory = false;
    end
    if showTheory
        if strcmpi(frfFnSource, 'beam'), fn = fnBeam; else, fn = fPk/sqrt(1 - zeta^2); end   % eq 7
        fT = theoryGrid(freqLim, fn, zeta, 4000);  r = fT/fn;
        magT = theoryCurve(fT, fn, zeta, Hpk);
        lagT = wrapLag(atan2(2*zeta*r, 1 - r.^2)*180/pi - 180*~disp_);
    end
    fprintf('Peak |H| = %.3f at %.2f Hz, phase lag there = %.1f deg\n', Hpk, fPk, lag(iPk));
    if showTheory, fprintf('Theory: fn = %.2f Hz (%s), zeta = %.4f\n', fn, frfFnSource, zeta); end

    [~, fName, fExt] = fileparts(file);  ir = f >= freqLim(1) & f <= freqLim(2);
    newFig(sprintf('Frequency response, %d in', beamLength), [100 80 1000 800]);
    tl = tiledlayout(2, 1, 'TileSpacing', 'compact');
    title(tl, sprintf('Frequency Response H(f) = Output / Input: %d in Beam, Experiment %s', beamLength, upper(experiment)));
    subtitle(tl, [fName fExt], 'Interpreter', 'none');
    nexttile;  hold on;
    plot(f(ir), magH(ir), '-', 'Color', blue, 'LineWidth', 1.2, 'DisplayName', 'Measured');
    plot(fPk, Hpk, 'kv', 'MarkerFaceColor', 'k', 'HandleVisibility', 'off');
    text(fPk, Hpk, sprintf('  %.2f Hz', fPk), 'VerticalAlignment', 'bottom');
    if showTheory
        plot(fT, magT, '--', 'Color', red, 'LineWidth', 1.5, 'DisplayName', sprintf('Theory (f_n = %.2f Hz, \\zeta = %.4f)', fn, zeta));
    end
    hold off;  lab('Frequency (Hz)', '|H(f)|', 'Magnitude of Frequency Response');  xlim(freqLim);  legend('Location', 'northeast');
    nexttile;  hold on;
    plot(f(ir), lag(ir), '.-', 'Color', blue, 'LineWidth', 1, 'MarkerSize', 8, 'DisplayName', 'Measured');
    if showTheory, plot(fT, lagT, '--', 'Color', red, 'LineWidth', 1.5, 'DisplayName', 'Theory'); end
    yline(lagLo + 180, ':', sprintf('%d^\\circ', lagLo + 180), 'HandleVisibility', 'off');
    xline(fPk, ':', sprintf('%.2f Hz', fPk), 'HandleVisibility', 'off');
    hold off;  lab('Frequency (Hz)', 'Phase lag (deg)', sprintf('Phase Lag of Output (%s) Relative to Input', lower(phaseRef)));
    xlim(freqLim);  ylim([lagLo, lagLo + 360]);  yticks(lagLo:45:lagLo + 360);  legend('Location', 'northeast');

case 'quad' % ===== hammer input/output vs. time and frequency (2 x 2) =====
    tFile = findDataFile(dataDir, 'fft', beamLength, experiment);
    fFile = findDataFile(dataDir, 'dat', beamLength, experiment);
    fprintf('Time series: %s\nFrequency  : %s\n', tFile, fFile);
```

```
    T = readmatrix(tFile, opt{:}, 'NumHeaderLines', 1);
    F = readmatrix(fFile, opt{:}, 'NumHeaderLines', 1);
    F = F(1:find(any(F(:,2:3) ~= 0, 2), 1, 'last'), :);     % drop zero padding above Nyquist
    if isempty(timeLim), timeLim = T([1 end], 1)'; end
    if isempty(quadFreqLim), quadFreqLim = [0 F(end,1)]; end
    [vHit, iHit] = max(T(:,3));
    inBand = find(F(:,1) >= max(quadFreqLim(1), 2) & F(:,1) <= quadFreqLim(2));  % skip DC drift below 2 Hz
    [aPk, k] = max(F(inBand,2));  fPk = F(inBand(k),1);
    fprintf('Hammer strike at t = %.4f s (peak %.3g %s)\n', T(iHit,1), vHit, strrep(voltUnit, '\times', 'x'));
    fprintf('Largest output peak in the frequency window: %.3g at %.2f Hz\n', aPk, fPk);
    [~, tn, te] = fileparts(tFile);  [~, fn_, fe] = fileparts(fFile);
    newFig(sprintf('Hammer test, %d in', beamLength), [80 80 1300 800]);
    tl = tiledlayout(2, 2, 'TileSpacing', 'compact', 'Padding', 'compact');
    title(tl, sprintf('Hammer Test: %d in Beam, Experiment %s', beamLength, upper(experiment)));
    %       x        y        colour  title                                        xlabel            ylabel      file     limits
    tiles = {T(:,1), T(:,3), red,  'Input (Hammer): Voltage vs. Time',          'Time (s)',       'Voltage',   [tn te], timeLim
             T(:,1), T(:,2), blue, 'Output (Beam): Voltage vs. Time',           'Time (s)',       'Voltage',   [tn te], timeLim
             F(:,1), F(:,3), red,  'Input (Hammer): Amplitude vs. Frequency',   'Frequency (Hz)', 'Amplitude', [fn_ fe], quadFreqLim
             F(:,1), F(:,2), blue, 'Output (Beam): Amplitude vs. Frequency',    'Frequency (Hz)', 'Amplitude', [fn_ fe], quadFreqLim};
    for p = 1:4
        nexttile;  plot(tiles{p,1}, tiles{p,2}, 'Color', tiles{p,3}, 'LineWidth', 1);  hold on;
        if p == 1, plot(T(iHit,1), vHit, 'kv', 'MarkerFaceColor', 'k');
                   text(T(iHit,1), vHit, sprintf('  strike, t = %.3f s', T(iHit,1)), 'VerticalAlignment', 'top'); end
        if p == 4, plot(fPk, aPk, 'kv', 'MarkerFaceColor', 'k');
                   text(fPk, aPk, sprintf('  %.2f Hz', fPk), 'VerticalAlignment', 'bottom'); end
        hold off;  lab(tiles{p,5}, [tiles{p,6} ' (' voltUnit ')'], tiles{p,4});
        subtitle(tiles{p,7}, 'Interpreter', 'none');  xlim(tiles{p,8});
    end

case 'sim' % ===== simulation runs: |H(f)| and phase lag, one figure per parameter =====
    simDir = findSimFolder(mainFolder, simFolder, simZip);
    if ~isempty(customRuns)   % label each chosen run with its case name from the groups table
        allRuns = [groups.runs];  allLabels = [groups.labels];  lbl = cell(size(customRuns));
        for i = 1:numel(customRuns)
            j = find(allRuns == customRuns(i), 1);
            if isempty(j), lbl{i} = sprintf('Run %d', customRuns(i)); else, lbl{i} = allLabels{j}; end
        end
        figs = struct('title', 'Selected Runs', 'runs', customRuns, 'labels', {lbl});
    elseif strcmpi(parameter, 'all'), figs = groups;
    else
        figs = groups(strcmpi({groups.key}, parameter));
        if isempty(figs), error('parameter must be ''length'', ''damping'', ''endmass'', ''material'' or ''all''.'); end
    end
    colors = lines(7);
    for g = 1:numel(figs)
        newFig(figs(g).title, [60 + 40*g, 60 + 30*g, 1000, 800]);
        tl = tiledlayout(2, 1, 'TileSpacing', 'compact');  title(tl, ['Simulation Frequency Response: ' figs(g).title]);
        axM = nexttile(1);  hold(axM, 'on');  axP = nexttile(2);  hold(axP, 'on');
        fprintf('\n===== %s =====\n%-4s %-20s %-14s %-12s %-16s\n', figs(g).title, 'Run', 'Case', 'Peak f [Hz]', 'Peak |H|', 'f at 90 deg [Hz]');
        fMax = 0;
        for i = 1:numel(figs(g).runs)
            X = figs(g).runs(i);  name = figs(g).labels{i};
            try, [fM, mag] = readRun(simDir, X, magType);  [fP, ph] = readRun(simDir, X, phaseType);
            catch err, warning('Run %d skipped: %s', X, err.message); continue; end
            lag = -ph;  [pk, k] = max(mag);                       % phase lag, 0 -> 180 deg
            f90 = interp1(lag + (1:numel(lag))'*1e-9, fP, 90);    % tiny ramp keeps x unique
            fMax = max([fMax; fM; fP]);  col = colors(mod(i-1, 7) + 1, :);
```

```
            plot(axM, fM, mag, '-', 'Color', col, 'LineWidth', 1.4, 'DisplayName', name);
            plot(axM, fM(k), pk, 'v', 'Color', col, 'MarkerFaceColor', col, 'HandleVisibility', 'off');
            plot(axP, fP, lag, '-', 'Color', col, 'LineWidth', 1.4, 'DisplayName', name);
            fprintf('%-4d %-20s %-14.2f %-12.4g %-16.2f\n', X, strrep(name, '_{end}', '_end'), fM(k), pk, f90);
        end
        if isempty(simFreqLim), xl = [0 fMax]; else, xl = simFreqLim; end
        axes(axM);  hold off;  lab('Frequency (Hz)', '|H(f)|', 'Magnitude of Response');
        if logMagnitude, set(axM, 'YScale', 'log'); end
        xlim(xl);  legend('Location', 'northeast');
        axes(axP);  hold off;  lab('Frequency (Hz)', 'Phase lag (deg)', 'Phase Angle');
        yline(90, ':', '90^\circ', 'HandleVisibility', 'off');
        xlim(xl);  ylim([0 180]);  yticks(0:30:180);  legend('Location', 'southeast');
    end

otherwise
    error('plotType must be ''amplitude'', ''natres'', ''frf'', ''quad'' or ''sim''.');
end

%% ==================== Local functions ====================
function newFig(name, pos)
    fig = figure('Name', name, 'NumberTitle', 'off');  fig.Position = pos;
end

function lab(xl, yl, tt)
    xlabel(xl);  ylabel(yl);  title(tt);  grid on;
end

function s = pm(v, u, nd)   % 'value +/- uncertainty' for the results table
    if isnan(v), s = '-'; else, s = sprintf(sprintf('%%.%df +/- %%.%df', nd, nd), v, u); end
end

function [y, uy] = propagate(fun, x, ux)   % y = fun(x); uy = RSS of each input moved by its uncertainty
    y = fun(x);  dy = zeros(size(x));
    for i = 1:numel(x), xi = x;  xi(i) = x(i) + ux(i);  dy(i) = fun(xi) - y; end
    uy = sqrt(sum(dy.^2));
end

function [fn, uFn] = beamNaturalFrequency(Lin, b)
% eqs 3, 4: k = 3EI/L^3, meq = 33/140*m + endMass; m = (L/totalLength)*totalMass, or rho*b*h*L
    useMass = ~isempty(b.totalMass);
    p = [Lin b.width b.thick b.E b.rho];  up = [b.uLength b.uWidth b.uThick b.uE b.uRho];
    if useMass, p = [p b.totalMass b.totalLength];  up = [up b.uTotalMass b.uTotalLength]; end
    [fn, uFn] = propagate(@fnOf, p, up);
    function f = fnOf(v)
        v(1:3) = v(1:3)*0.0254;                                  % in -> m
        if useMass, m = v(1)/(v(7)*0.0254)*v(6); else, m = v(5)*v(2)*v(3)*v(1); end
        f = sqrt(3*v(4)*v(2)*v(3)^3/12/v(1)^3/(33/140*m + b.endMass))/(2*pi);
    end
end

function [zeta, uZeta, src] = dampingFromHammer(dataDir, L, expLetter, fnBeam, nCycles, nSkip)
% Log decrement of the hammer time series; falls back to the other experiment letter.
    zeta = NaN;  uZeta = NaN;  src = '';
    for e = {expLetter, char('a' + 'b' - expLetter)}
        try, file = findDataFile(dataDir, 'fft', L, e{1}); catch, continue; end
        M = readmatrix(file, 'FileType', 'text', 'NumHeaderLines', 1, 'Delimiter', '\t');
        [z, uz] = logDecrement(M(:,1), M(:,2), M(:,3), fnBeam, nCycles, nSkip);
```

```
        if ~isnan(z) && z > 0
            zeta = z;  uZeta = uz;  [~, n, x] = fileparts(file);  src = ['log decrement of ' n x];
            if ~strcmp(e{1}, expLetter)
                fprintf('Note: log decrement of experiment %s was not usable; used %s.\n', upper(expLetter), [n x]);
            end
            return;
        end
    end
end

function [zeta, uZeta] = logDecrement(t, y, hammer, fnBeam, nCycles, nSkip)
% FFT band-pass 0.5-1.5 x fnBeam, positive peaks after the strike, eqs 5 and 6.
% u_zeta from the scatter of the single-cycle decrements.
    zeta = NaN;  uZeta = NaN;  N = numel(y);
    Y = fft(y - mean(y));  k = (0:N-1)';  fr = min(k, N-k)/(N*mean(diff(t)));
    Y(fr < 0.5*fnBeam | fr > 1.5*fnBeam) = 0;  yf = real(ifft(Y));
    [~, iHit] = max(hammer);
    pk = find(yf(2:end-1) > yf(1:end-2) & yf(2:end-1) >= yf(3:end) & yf(2:end-1) > 0) + 1;
    pk = pk(pk > iHit);  pk = pk(nSkip+1:end);  n = min(nCycles, numel(pk) - 1);
    if n < 2, return; end
    x = yf(pk(1:n+1));
    D = log(x(1)/x(n+1))/n;                                      % eq 6
    zeta = D/sqrt((2*pi)^2 + D^2);                               % eq 5
    uZeta = std(log(x(1:n)./x(2:n+1)))/sqrt(n)*(2*pi)^2/((2*pi)^2 + D^2)^1.5;
end

function zeta = halfPowerZeta(f, A, iPk)   % half-power (-3 dB) bandwidth around the peak
    h = A(iPk)/sqrt(2);  i = iPk;  j = iPk;
    while i > 1 && A(i) > h, i = i - 1; end
    while j < numel(A) && A(j) > h, j = j + 1; end
    fLo = f(i) + (h - A(i))*(f(i+1) - f(i))/(A(i+1) - A(i));
    fHi = f(j-1) + (h - A(j-1))*(f(j) - f(j-1))/(A(j) - A(j-1));
    zeta = (fHi - fLo)/(2*f(iPk));
end

function fq = theoryGrid(lim, fnSet, zSet, n)   % even grid plus extra points around each fn
    fq = linspace(max(lim(1), 1e-3), lim(2), n)';
    for i = 1:numel(fnSet), fq = [fq; fnSet(i)*(1 + (-5:0.01:5)'*zSet(i))]; end %#ok<AGROW>
    fq = unique(fq(fq >= lim(1) & fq <= lim(2) & fq > 0));
end

function [theo, scale] = theoryCurve(fq, fn, zeta, Amax)   % R*w^2 scaled so its max = Amax
    r = fq/fn;  Rw = (2*pi*fq).^2./sqrt((1 - r.^2).^2 + (2*zeta*r).^2);
    scale = Amax/max(Rw);  theo = scale*Rw;
end

function [f, A] = readShaker(file)   % col 2 frequency, col 3 output; drop repeated-frequency rows
    M = readmatrix(file, 'FileType', 'text', 'NumHeaderLines', 0, 'Delimiter', '\t');
    [~, keep] = unique(M(:,2), 'first');  keep = sort(keep);
    if numel(keep) < size(M,1), fprintf('Removed %d repeated-frequency row(s).\n', size(M,1) - numel(keep)); end
    f = M(keep,2);  A = M(keep,3);
end

function tf = isTimeFile(file)   % header line says the first column is time
    fid = fopen(file, 'r');  header = fgetl(fid);  fclose(fid);
    tf = startsWith(strtrim(lower(header)), 'time');
end
```

```
function dataDir = findDataFolder(mainFolder, zipName, datasetFolder, groupFolder)
% <mainFolder>\<datasetFolder>\<groupFolder>, extracting the zip first if needed.
    if ~isfolder(mainFolder), error('Main folder not found: %s', mainFolder); end
    dataDir = fullfile(mainFolder, datasetFolder, groupFolder);
    if isfolder(dataDir), return; end
    zipPath = fullfile(mainFolder, zipName);
    if ~isfile(zipPath)   % accept other spellings, e.g. 'Vibrations_datasets.zip'
        z = dir(fullfile(mainFolder, '*.zip'));  z = z(contains(lower({z.name}), 'vibration'));
        if isempty(z), error('Neither the folder %s nor a vibrations zip file was found in %s.', ...
                             fullfile(datasetFolder, groupFolder), mainFolder); end
        zipPath = fullfile(mainFolder, z(1).name);
    end
    fprintf('Extracting %s ...\n', zipPath);  unzip(zipPath, mainFolder);
    if isfolder(dataDir), return; end
    d = dir(fullfile(mainFolder, '**'));  d = d([d.isdir] & strcmpi({d.name}, groupFolder));
    if isempty(d), error('Folder %s not found inside %s.', groupFolder, zipPath); end
    dataDir = fullfile(d(1).folder, d(1).name);
end

function filePath = findDataFile(dataDir, fileType, L, expLetter)
% M003 names: 10_8_2026m003_hammer_16_adat.txt / _afft.txt, Shaker16_inch_A.txt
% (other groups' spellings such as hammer16inadat or shaker16inchA also work)
    files = dir(fullfile(dataDir, '*.txt'));  names = {files.name};
    if strcmp(fileType, 'shaker'), pat = sprintf('^shaker?_?%d\\D*%s\\.txt$', L, expLetter);
    else, pat = sprintf('hammer\\D*%d\\D.*%s%s\\.txt$', L, expLetter, fileType); end
    match = names(~cellfun(@isempty, regexpi(names, pat, 'once')));
    if isempty(match), error('No %s file for %d in, experiment %s, found in %s.', fileType, L, upper(expLetter), dataDir); end
    if numel(match) > 1   % prefer the shortest name (Shaker16_inch_A over Shaker16NEW_inch_A)
        [~, o] = sort(cellfun(@numel, match));  match = match(o);
        fprintf('Several files match; using the first:\n');  fprintf('  %s\n', match{:});
    end
    filePath = fullfile(dataDir, match{1});
end

function dataDir = findSimFolder(mainFolder, folder, zipName)
% <mainFolder>\<folder>, extracting <zipName> into it if the folder does not exist yet.
    if ~isfolder(mainFolder), error('Main folder not found: %s', mainFolder); end
    dataDir = fullfile(mainFolder, folder);
    if isfolder(dataDir), return; end
    zipPath = fullfile(mainFolder, zipName);
    if ~isfile(zipPath)   % accept other spellings, e.g. 'Lab_Data_V.zip'
        z = dir(fullfile(mainFolder, '*.zip'));  z = z(contains(lower(regexprep({z.name}, '[_ ]', '')), 'labdatav'));
        if isempty(z), error('Neither the folder "%s" nor a Lab Data V zip file was found in %s.', folder, mainFolder); end
        zipPath = fullfile(mainFolder, z(1).name);
    end
    fprintf('Extracting %s ...\n', zipPath);  unzip(zipPath, dataDir);
    t = dir(fullfile(dataDir, '**', '*james*.txt'));   % the zip may contain its own folder
    if ~isempty(t), dataDir = t(1).folder; end
end

function [f, v] = readRun(dataDir, X, Y)
% Run X, type Y ('dat'/'fft'): col 1 frequency, col 2 value. Matches 10_8_2026jamesXvY.txt,
% also with a space before X or an extra digit in the date ('10_8_20261james 3vdat.txt').
    files = dir(fullfile(dataDir, '*.txt'));  names = {files.name};
    match = names(~cellfun(@isempty, regexpi(names, sprintf('james\\s*%d\\s*v\\s*%s\\.txt$', X, Y), 'once')));
    if isempty(match), error('No file for run %d, type ''%s'' in %s.', X, Y, dataDir); end
    if numel(match) > 1, fprintf('Several files for run %d %s; using %s\n', X, Y, match{1}); end
```

```
    M = readmatrix(fullfile(dataDir, match{1}), 'FileType', 'text', 'NumHeaderLines', 1, 'Delimiter', '\t');
    f = M(:,1);  v = M(:,2);
end
```